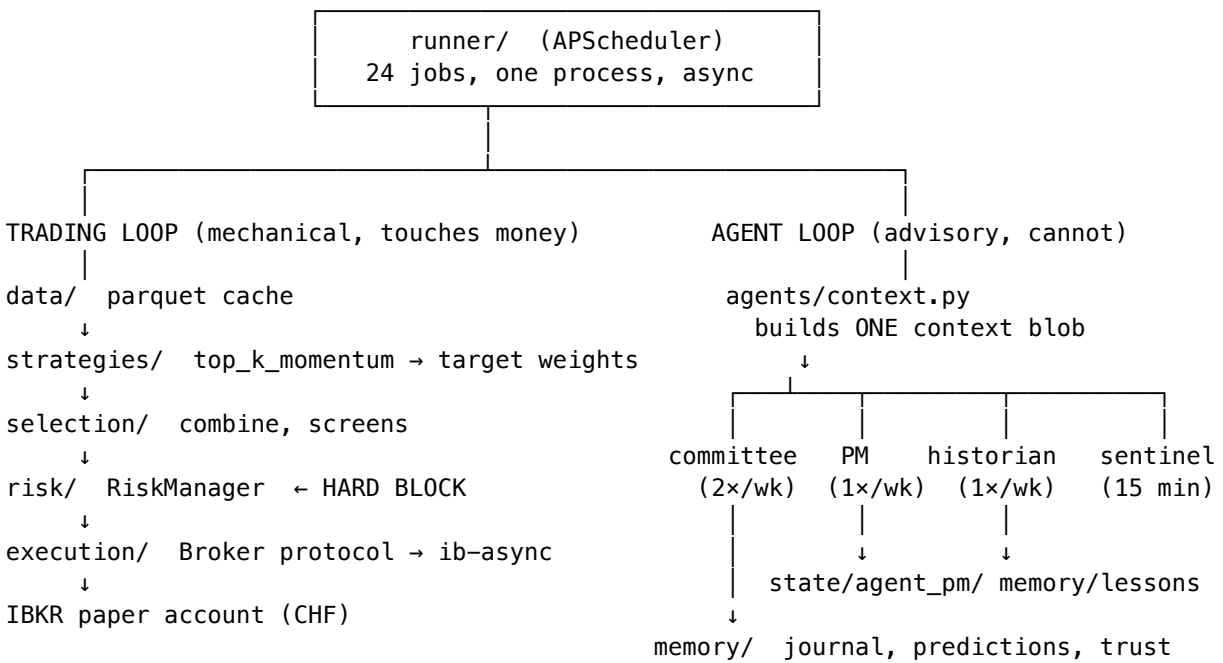
Architecture map

Two independent loops



The only connection between the loops is read-only: `build_context` reads the traded book's snapshot so the agents can discuss it. No agent module imports `execution/`, and `tests/copilot/test_copilot.py` asserts that as a spawned-subprocess import check.

Packages, by size and job

Package	LOC	What it actually does
`runner/`	4067	Scheduler, cycle orchestration, order state machine, kill switches, holds
`runtime/`	3937	Monitors that write JSON state files: macro dial, options/vol, HMM regime, style, news, econ
`bot/`	3153	Telegram: ~40 commands, keyboards, notifier, command registry
`agents/`	2531	committee, pm, historian, guards, context, llm routing, candidates
`copilot/`	1954	Conversational Q&A over the desk's own state; mandates; thread memory
`dashboard/`	1934	Read-only HTTP view on :8787, six tabs
`strategies/`	1712	Strategy protocol + momentum/mean-reversion/pairs/risk-parity library
`execution/`	1640	Broker protocol, IBKR adapter, simulator
`selection/`	1575	Ranking, screens, combination, hedge overlay
`memory/`	928	The permanent spine: SQLite + markdown lesson cards
`reporting/`, `data/`, `core/`, `risk/`, `backtest/`, `regime/`, `portfolio/`	~4000	Support

Schedule (all UTC)

When	Job	Touches money?
every 5s	`command_processor`	via approved orders
every 30s	`trigger_watcher`, `committee_trigger`, `agent_pm_trigger`	no
every 60s	`watchdog`	kill switch only
Mon–Fri :07	`memory_grader` (hourly at minute 7)	no
Mon–Fri 11:00	`agent_committee`	no
Mon–Fri 13:30	`macro_monitor`	no
Mon–Fri 13:40	`news_watch`	no
Mon–Fri 13:00–20:00 /15min	`sentinel`	no — alerts only
Mon 14:30	**`agent_pm`**	no — sim book
Mon–Fri 15:45, 19:45	`risk_advisor`	no
Mon–Fri 20:10 / 20:20	`style_advisor`, `market_watch`	no
Mon–Fri 21:05	**`cycle`** — the actual rebalance	YES
Mon–Fri 21:15	`agent_pm_mark`	no
22:15	`snapshot_refresh`	no
22:30	`daily_summary`, prediction grading	no
Fri 22:45	`historian`	no
Sun 12:00	`econ_watch`	no

Where state lives

```
state/
  runner.db          account snapshots, equity curve      ← the traded book
  orders.db          order + fill ledger
  halt.json          kill-switch state
  holds.json         operator /hold pins
  agent_pm/
    portfolio.json   the SIM book: cash, holdings, marks, history
    last_run.json    last PM decision + rationale
  memory/
    memory.db        journal, episodes, lessons, predictions, source_trust, shadow
    lessons/*.md     Obsidian-compatible lesson cards
  sentinel.json, macro_monitor.json, options_monitor.json, advisor.json, ...
data/parquet/
  equity/<SYM>/1D.parquet      ~516 symbols
  etf/<SYM>/1D.parquet
```

Note the asset-class split: ETFs live under **`etf/`**, stocks under **`equity/`**, and some older files are `1d.parquet` not `1D.parquet`. Use `runtime.portfolio_stats._read_close`, which handles both — a first cut of the candidate ladder hardcoded `equity/1D` and silently returned an empty ladder against a fully populated cache.

Deployment

Single Docker image `trading-agent:latest`, five services sharing it: `trader` (the runner), `bot`, `dashboard`, plus `ib-gateway` and `scheduler` (ofelia, for prune/vacuum cron). A `mirror` profile adds a read-only live-account watcher. Code is baked into the image — source changes need `docker compose build trader`; `config/` is bind-mounted read-only so YAML edits need only a `git pull`.